AI-BASED STUDENT PERFORMANCE PREDICTION SYSTEM

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA1711 Artificial Intelligence

to the award of the degree of

BACHELOR OF ENGINEERING

IN

CSE(DS)

Submitted by

T. Niranjan Kumar (192473007)

P. Shashank (192424355)

Under the Supervision of

DR. K. Krishnakumar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September-2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, **T. Niranjan Kumar, P. Shashank** Department of the **CSE(DS),** Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **"AI-BASED STUDENT PERFORMANCE PREDICTION SYSTEM"** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics

Place: Thandalam, Chennai

Date: 22-09-2025

Signature of the Students with Names

T. Niranjan Kumar (192473007)

P. Shashank (192424355)

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled **"AI-BASED STUDENT PERFORMANCE PREDICTION SYSTEM"** has been carried out by, K. Vijay Bhaskar Reddy , K. Sumanth Kumar Reddy under the supervision of DR. J. VELMURUGAN and DR. S. PANDIRAJ is submitted in partial fulfilment of the requirements for the current semester of the B.Tech program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE	SIGNATURE
Dr. Anasuya	**DR. K. Krishnakumar**
Program Director	
Department Name (CSE)	Associate Professor of CSE
Saveetha School of Engineering	Saveetha School of Engineering
SIMATS	SIMATS

Submitted for the Project work Viva-Voce held on ___________________________________
.

INTERNAL EXAMINER EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, **Dr. Anasuya** for his continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide **DR. K. Krishnakumar** for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

T. Niranjan Kumar (192473007)

P. Shashank (192424355)

ABSTRACT

Student performance prediction is an important application of Artificial Intelligence and Machine Learning that can support effective academic monitoring and early identification of students who may require additional support. Traditional methods mainly depend on manual observation of attendance, marks, assignments, and previous academic results, which may make early identification of weak students difficult. This project, titled “AI-Based Student Performance Prediction System”, focuses on developing a web-based machine learning system to predict student performance using academic and behavioral factors.

The system uses student parameters such as attendance, internal marks, assignment score, study hours, previous result, sleep hours, extracurricular activities, internet access, and parent support. The data is preprocessed to handle missing values and prepare it for machine learning. Random Forest and XGBoost algorithms are trained and compared using standard evaluation measures such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The system also supports individual and batch prediction while maintaining the StudentID and Name of each student. In addition to performance prediction, it identifies students who may be at risk and provides possible contributing factors such as low attendance or low internal marks.

The proposed system provides an interactive web-based platform for dataset exploration, model comparison, performance analysis, individual prediction, batch prediction, and result visualization. The system helps teachers and academic advisors understand student performance and identify students who may need early attention. The project demonstrates how machine learning can be applied to education to support data-driven decision-making, early risk identification, and improved academic monitoring.

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 Background Information

Student academic performance is an important aspect of the education system, as it helps educational institutions understand the learning progress of students. Student performance can be influenced by various academic and behavioral factors such as attendance, internal marks, assignment scores, study hours, previous academic results, and other related factors. Monitoring these factors can help teachers understand the academic condition of students and provide appropriate support when required.

Traditional methods of monitoring student performance mainly depend on manual observation, examination results, attendance records, and teacher analysis. Although these methods are useful, they may make it difficult to identify weak-performing students at an early stage. With the development of Artificial Intelligence and Machine Learning, student data can be analyzed automatically to discover patterns and predict possible performance levels.

The proposed **AI-Based Student Performance Prediction System** is developed to address this problem by using Machine Learning techniques to predict student academic performance. The system analyzes multiple student-related parameters and provides a performance prediction. It also supports early identification of students who may be at risk and provides possible factors contributing to their predicted performance.

1.2 Project Objectives

The primary objectives of the Disaster Response Coordination Platform include:

1. **Student Data Preprocessing:** To clean and preprocess student academic and behavioral data.
2. **Performance Prediction:** To predict student performance using Machine Learning algorithms.
3. **Model Comparison:** To compare Random Forest and XGBoost algorithms based on actual evaluation results.
4. **Risk Identification:** To identify weak and at-risk students at an early stage.
5. **Student Identification:** To maintain StudentID and Name along with prediction results.

1.3 Significance

The significance of the proposed system is to provide an automated and efficient approach for analyzing student academic performance. By considering multiple factors instead of depending on a single parameter, the system can provide a more useful view of the student's academic condition.

The system can help teachers and academic advisors identify students who may require additional attention before their performance becomes poor. The prediction results and contributing factors can support timely guidance and academic intervention. The system also provides model evaluation and comparison, making the prediction process easier to understand.

1.4 Scope

The scope of the project encompasses:

- Student academic and behavioral data preprocessing.
- Prediction of student performance using Machine Learning.
- Comparison of Random Forest and XGBoost models.
- Early identification of weak and at-risk students.
- Individual and batch student prediction.
- Maintaining StudentID and Name in prediction results.
- Visualization of student performance and model results.
- Upload and processing of a limited dataset of up to 500 students..

1.5 Methodology Overview

The methodology adopted for this project involves data preprocessing, Machine Learning model development, model evaluation, and web-based prediction. The labelled student dataset is first processed by handling missing values and preparing the required numerical and categorical features. The data is then divided into training and testing sets for model development and evaluation.

Random Forest and XGBoost algorithms are trained using the training data and evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The actual test results are used to compare the models and select the better-performing model. The selected model is then used for predicting new student records.

The system also allows users to upload a new dataset without the **Performance** column. The uploaded student records are processed using the same preprocessing steps, and predictions are generated in batch. The final results include **StudentID, Name, Predicted Performance, Risk Level, and Contributing Factors**, helping users identify students who may need early academic support.

CHAPTER 2

PROBLEM IDENTIFICATON & ANALYSIS

2.1 Description of the Problem

- **Manual Performance Analysis:** Student performance is often analyzed manually using marks, attendance, and teacher observations, which can be time-consuming when many students are involved.
- **Difficulty in Early Identification:** Weak or at-risk students may not be identified early because different academic and behavioral factors are not always analyzed together.
- **Multiple Performance Factors:** Student performance depends on several factors such as attendance, internal marks, assignment scores, study hours, previous results, and other behavioral parameters.
- **Data Quality Issues:** Student datasets may contain missing values, inconsistent values, or different column formats, which can affect the prediction process.
- **Lack of Automated Prediction:** Traditional approaches do not provide an automated Machine Learning-based prediction system that can analyze new student records and generate performance results.

2.2 Evidence of the Problem

- The need for an automated student performance prediction system can be observed from the difficulties involved in analyzing multiple student parameters manually. Attendance, internal marks, assignment scores, study hours, previous results, and other factors provide useful information about a student's academic condition, but analyzing them together for every student requires considerable effort.
- The problem becomes more important when new student records need to be analyzed. A new prediction dataset normally does not contain the actual Performance value because the purpose of the system is to predict it. Therefore, model accuracy cannot be calculated directly from the new uploaded dataset. The system must evaluate its models separately using labelled test data and then use the selected model for predicting new students.

2.3 Stakeholders

- The major stakeholders of the proposed system are students, teachers, academic advisors, and educational institutions. Students can benefit from early information about their predicted performance and risk level. Teachers can use the prediction results to identify students who may require additional academic attention and guidance.
- Academic advisors can use the system to understand possible factors affecting student performance and provide suitable support. Educational institutions can use the overall analysis for academic monitoring and planning. The system can therefore support different stakeholders by providing student performance information in a simple and understandable manner.

2.4 Supporting Data/Research

- Machine Learning provides suitable techniques for analyzing historical student records and predicting performance based on previously observed patterns. The proposed system uses **Random Forest and XGBoost** classification algorithms to compare different machine learning approaches for student performance prediction.
- The system evaluates the trained models using standard measures such as **Accuracy, Precision, Recall, F1-Score, and Confusion Matrix**. Proper preprocessing, including handling missing values and preparing numerical and categorical features, is also important for reliable prediction. These techniques provide the foundation for developing the proposed student performance prediction and early risk identification system.

CHAPTER 3

SOLUTION DESIGN & IMPLEMENTATION

3.1 Development & Design Process

The development of the **AI-Based Student Performance Prediction System** involves a systematic process of data preparation, machine learning model development, evaluation, and web application integration. The system is designed to accept student datasets, preprocess the data, train machine learning models, compare their performance, and use the best-performing model for predicting new student records.

The development process begins with preparing the labelled training dataset containing student identification, academic, behavioral, and support-related parameters along with the Performance target. Missing values and inconsistent data are handled during preprocessing. The processed data is then divided into training and testing sets. Random Forest and XGBoost algorithms are trained using the training data and evaluated using standard performance metrics.

After selecting the better-performing model, the trained model and preprocessing components are saved for future predictions. The web application allows users to upload a new student dataset without the Performance column. The uploaded records are processed using the same preprocessing pipeline and predictions are generated in batch while maintaining StudentID and Name.

3.2 Tools & Technologies Used

- **Python:** Used for data processing and Machine Learning implementation.
- **Pandas:** Used for loading, cleaning, and processing student datasets.
- **NumPy:** Used for numerical operations and data manipulation.
- **Scikit-learn:** Used for preprocessing, Random Forest, and model evaluation.
- **XGBoost:** Used for gradient boosting-based performance prediction.
- **Flask:** Used to develop the web application backend.
- **HTML & CSS:** Used to design the user interface.
- **JavaScript:** Used for interactive dashboard functionality.
- **Matplotlib/Charts:** Used for data visualization and performance analysis.
- **Joblib:** Used to save and load trained models and preprocessing objects.

3.3 Solution Overview

The proposed system accepts student records containing identification, academic, behavioral, and support-related parameters. The dataset includes fields such as StudentID, Name, Attendance, Internal Marks, Assignment Score, Study Hours, Previous Result, Sleep Hours, Extra Curricular, Internet Access, Parent Support, and Performance. StudentID and Name are maintained for identifying students, while the relevant input parameters are used by the Machine Learning models.

The system first preprocesses the training data by handling missing values and preparing numerical and categorical features. The labelled data is divided into training and testing portions. Random Forest and XGBoost models are then trained and evaluated using the unseen testing data. Accuracy, Precision, Recall, F1-Score, and Confusion Matrix are used to compare the models and select the better-performing model.

For new student prediction, the user can upload a dataset without the Performance column. The system validates the uploaded data, applies the same preprocessing steps, and uses the selected model to generate predictions. The final output contains StudentID, Name, Predicted Performance, Risk Level, and possible Contributing Factors.

3.4 Engineering Standards Applied

- **Modular Design:** Separates data processing, training, evaluation, and prediction operations.
- **Input Validation:** Checks uploaded datasets for required columns and valid values.
- **Error Handling:** Provides clear messages when invalid or incomplete data is uploaded.
- **Data Separation:** Keeps model training and new student prediction processes separate.
- **Consistent Preprocessing:** Uses the same preprocessing approach for training, testing, and prediction data.
- **Model Evaluation:** Evaluates models using unseen test data instead of training data.
- **Maintainability:** Keeps the system organized for easier modification and future development.

3.5 Solution Justification

The proposed solution uses Machine Learning because student performance depends on several academic and behavioral factors that can be analyzed together. Random Forest and XGBoost are selected because they provide two different ensemble-based approaches for classification and allow their actual performance to be compared before selecting the final model.

The system also separates model evaluation from new student prediction. This is important because the uploaded prediction dataset does not contain the actual Performance value. Therefore, its accuracy cannot be calculated at the time of prediction. Instead, the model accuracy is calculated using labelled unseen test data. The addition of early risk identification and contributing factors makes the system more useful for teachers and academic advisors by helping them identify students who may need timely support.

CHAPTER 4

RESULT & RECOMMENDATIONS

4.1 Evaluation of Results

The implementation of the **AI-Based Student Performance Prediction System** demonstrates the ability of Machine Learning techniques to analyze student academic and behavioral data and predict performance categories. The system preprocesses the student dataset, trains Random Forest and XGBoost models, and evaluates their performance using unseen test data. The evaluation provides measurable results that help in selecting the better-performing model.

During model evaluation, the system calculates **Accuracy, Precision, Recall, and F1-Score** and generates a Confusion Matrix to understand the classification results. The model comparison allows the system to identify which algorithm performs better on the given training and testing data. The accuracy shown in the system is calculated from actual test predictions and is not manually assigned.

The batch prediction functionality successfully allows new student records to be uploaded without the Performance column. The system processes the uploaded records and generates predictions while maintaining the **StudentID and Name** of each student. The output also provides the predicted performance category, risk level, and possible contributing factors.

The overall results indicate:

- Successful preprocessing and handling of student data.
- Successful training and comparison of Random Forest and XGBoost.
- Actual evaluation of models using unseen test data.
- Prediction of student performance for new records.
- Early identification of weak and at-risk students.
- Identification of possible factors affecting student performance.
- Support for batch prediction of up to 500 student records.

Resources

Resource	Total	Used	Available	Actions	
Water	3800	2188	1612	+100	-100
Medical Kits	5500	4589	911	+100	-100
Food	4800	986	3814	+100	-100
Blankets	3400	3204	196	+100	-100
Oxygen	4900	4890	10	+100	-100
Rescue Tools	5800	5426	374	+100	-100

Fig No.1 Resource Management

4.2 Challenges Encountered

- **Missing Values:** Some datasets may contain missing values, which can cause errors if they are not handled during preprocessing.
- **Column Mismatch:** Uploaded datasets may use different column names from those expected by the trained model.
- **Prediction Dataset:** New student data does not contain the Performance value, so accuracy cannot be calculated from the uploaded prediction data.
- **Batch Processing:** Processing a large number of student records individually can increase prediction time.
- **Model Evaluation:** Ensuring that accuracy represents unseen test data rather than training data required careful separation of the data flow.

4.3 Possible Improvements

- **Larger Dataset:** Train the models using more diverse student records.
- **Hyperparameter Tuning:** Improve model performance by optimizing model parameters.
- **Explainable AI:** Provide clearer explanations for individual predictions.
- **Personalized Suggestions:** Give students recommendations based on their weak areas.
- **Real-Time Data:** Integrate live attendance and academic records.
- **Mobile Application:** Develop a mobile version for easier access.
- **Cloud Deployment:** Deploy the system online for wider institutional.

4.4 Recommendations

- Use a reliable and properly labelled dataset for model training and evaluation.
- Handle missing and inconsistent values before applying Machine Learning algorithms.
- Keep the training, testing, and new prediction datasets separate.
- Use actual test-set results when reporting model accuracy.
- Maintain StudentID and Name for identification without using them as prediction features.
- Teachers and academic advisors should verify important predictions before taking academic decisions.
- Retrain the model periodically when new student data becomes available.

CHAPTER 5

REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT

5.1 Key Learning Outcomes

5.1.1 Academic Knowledge

- The project helped me strengthen my understanding of Artificial Intelligence and Machine Learning concepts. I gained practical knowledge about supervised learning, classification, data preprocessing, model training, and model evaluation. I also understood how different academic and behavioral factors can be used to predict student performance.

5.1.2 Technical Skills

- During the development of the project, I gained practical experience in Python, Pandas, NumPy, Scikit-learn, XGBoost, Flask, HTML, CSS, and JavaScript. I learned how to preprocess datasets, train Machine Learning models, compare their performance, save trained models, and integrate them into a web-based application.

5.1.3 Problem-Solving & Critical Thinking

- The project improved my problem-solving and critical-thinking abilities through practical challenges involving missing values, incorrect column names, dataset validation, model prediction, and batch processing. I learned to identify the actual cause of an error and modify the data processing or application workflow instead of changing only the displayed result.

5.2 Challenges Encountered and Overcome

- During the development of the system, several challenges were encountered while working with student datasets and Machine Learning models. Handling missing values and inconsistent data formats required proper preprocessing so that the model could process the data correctly.

- Another important challenge was understanding the difference between model evaluation data and new prediction data. The uploaded student prediction dataset does not contain the actual Performance value, so it cannot be used to calculate accuracy. This was overcome by keeping the labelled training and testing data separate from the new student prediction dataset.
- The project also required maintaining StudentID and Name in the final prediction results without using them as prediction features. Batch prediction was implemented so that multiple student records could be processed efficiently.

5.3 Applications of Engineering Standards

- Ethical considerations are important when working with student academic information. The system should protect student information and use the collected data only for appropriate academic purposes. StudentID and Name are used for identification in the output, while unnecessary personal information is avoided.
- The prediction generated by the system should be treated as decision-support information rather than a final judgement about a student. Teachers and academic advisors should review the prediction and consider other relevant factors before taking important academic decisions.

5.4 Applications of Ethical Standards

- Ethical considerations are important when working with student academic information. The system should protect student information and use the collected data only for appropriate academic purposes. StudentID and Name are used for identification in the output, while unnecessary personal information is avoided.
- The prediction generated by the system should be treated as decision-support information rather than a final judgement about a student. Teachers and academic advisors should review the prediction and consider other relevant factors before taking important academic decisions.

5.5 Insights into the Industry

- The project provided an understanding of how Artificial Intelligence and Machine Learning can be integrated into real-world software applications. I learned that developing an AI system involves more than selecting an algorithm. Data quality, preprocessing, model evaluation, application development, user interaction, error handling, and result interpretation are also important.
- The project also provided practical knowledge about developing web-based Machine Learning applications. The integration of a trained model with a Flask application showed how Machine Learning can be converted into a usable software solution for real-world users.

5.6 Conclusion of Personal Development

- The development of the AI-Based Student Performance Prediction System improved my technical knowledge, problem-solving ability, and understanding of Machine Learning applications. I gained practical experience in developing an end-to-end system from dataset preprocessing to model training, evaluation, prediction, and web-based result presentation.
- Overall, the project helped me understand how Artificial Intelligence can be applied to solve practical problems in education. It also improved my confidence in developing, testing, debugging, and explaining a complete Machine Learning-based software application.

CHAPTER 6

CONCLUSION

The **AI-Based Student Performance Prediction System** developed in this project demonstrates how Artificial Intelligence and Machine Learning can be used to support student academic performance analysis. The system combines student academic and behavioral parameters such as attendance, internal marks, assignment score, study hours, previous result, and other relevant factors to predict student performance.

The system uses **Random Forest and XGBoost** algorithms and compares their actual performance using standard evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. The model evaluation is performed using unseen test data, ensuring that the reported results represent the actual predictive performance of the trained models. The better-performing model is selected for predicting new student records.

The system also supports **individual and batch prediction** for new student datasets. StudentID and Name are maintained in the results so that each prediction can be clearly identified. Along with predicted performance, the system provides a risk level and possible contributing factors, helping teachers and academic advisors identify students who may require early academic attention.

In conclusion, the proposed system provides a practical and user-friendly solution for student performance prediction and early risk identification. The project demonstrates the effective integration of Machine Learning algorithms with a web-based application and provides a foundation for future improvements such as personalized recommendations, Explainable AI, real-time data integration, mobile applications, and cloud deployment.

REFERENCES

1. Breiman, L. (2001). Random Forests. *Machine Learning, 45,* 5–32.
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
3. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.
4. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*.
5. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
6. Raschka, S., Liu, Y., & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing.
7. Molnar, C. (2022). *Interpretable Machine Learning*. A Guide for Making Black Box Models Explainable.
8. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
9. Python Software Foundation. *Python Documentation*. Python Software Foundation.
10. XGBoost Developers. *XGBoost Documentation*. Documentation for gradient boosting and machine learning.

APPENDICES

Appendix A: Dataset Parameters

The dataset used in the proposed system contains student identification, academic, behavioral, and support-related information. The labelled training dataset also contains the Performance column, which is used as the target variable for Machine Learning model training and evaluation.

Dataset Parameters:

- **StudentID – Unique identification number of the student.**
- **Name – Name of the student.**
- **Attendance – Student attendance percentage.**
- **Internal_Marks – Marks obtained in internal examinations.**
- **Assignment_Score – Score obtained in assignments.**
- **Study_Hours – Average study hours of the student.**
- **Previous_Result – Previous academic performance.**
- **Sleep_Hours – Average sleeping hours.**
- **Extra_Curricular – Participation in extracurricular activities.**
- **Internet_Access – Availability of internet access.**
- **Parent_Support – Level of parental academic support.**

Appendix B: Sample Student Dataset

The sample student dataset represents the type of records used by the system for performance prediction. StudentID and Name are maintained to identify each student, while the remaining parameters provide the information required for prediction.

StudentID	Name	Attendance	Internal Marks	Assignment Score	Study Hours
S001	Arjun Reddy	92	88	90	6
S002	Kavya Sharma	85	78	82	5
S003	Ravi Kumar	68	55	60	3
S004	Divya Reddy	95	91	94	7
S005	Rahul Verma	62	48	52	2

Appendix C: Prediction Output

The prediction output contains the original student identification details along with the results generated by the trained Machine Learning model. The system does not require the Performance column in the new prediction dataset because Performance is the value that needs to be predicted.

The output contains:

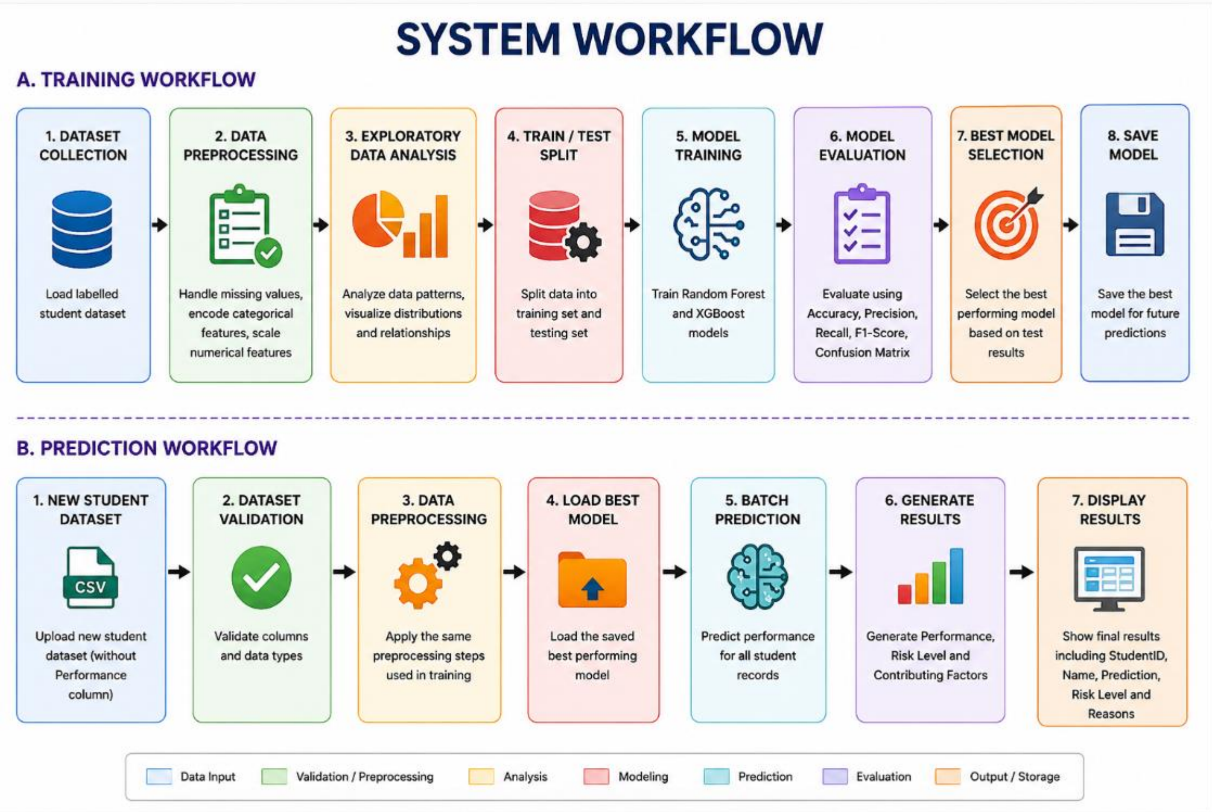
- StudentID
- Name
- Predicted Performance

• Risk Level
• Contributing Factors

For example, a student record can be displayed as:

StudentID: S103
Name: Arjun Reddy
Prediction: Poor
Risk: High
Reasons: Low Attendance + Low Internal Marks

Appendix D: Deployment Workflow (Flowchart)



Appendix E: CODE (Simplified)

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)
```

```python
# ----------------------------------------------------
# 1. LOAD DATASET
# ----------------------------------------------------

df = pd.read_csv("student_performance.csv")

print("Dataset Shape:", df.shape)
print(df.head())


# ----------------------------------------------------
# 2. HANDLE MISSING VALUES
# ----------------------------------------------------

# Numerical columns
num_cols = df.select_dtypes(
    include=["int64", "float64"]
).columns

df[num_cols] = df[num_cols].fillna(
    df[num_cols].median()
)

# Categorical columns
cat_cols = df.select_dtypes(
    include=["object"]
).columns

df[cat_cols] = df[cat_cols].fillna("Unknown")


# ----------------------------------------------------
# 3. SEPARATE FEATURES AND TARGET
# ----------------------------------------------------

X = df.drop(
    columns=["Performance", "StudentID", "Name"]
)

y = df["Performance"]


# ----------------------------------------------------
# 4. TRAIN-TEST SPLIT
# ----------------------------------------------------
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)


# --------------------------------------------------
# 5. RANDOM FOREST MODEL
# --------------------------------------------------

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

rf_predictions = rf_model.predict(X_test)


# --------------------------------------------------
# 6. XGBOOST MODEL
# --------------------------------------------------

xgb_model = XGBClassifier(
    n_estimators=100,
    random_state=42
)

xgb_model.fit(X_train, y_train)

xgb_predictions = xgb_model.predict(X_test)


# --------------------------------------------------
# 7. MODEL EVALUATION
# --------------------------------------------------

rf_accuracy = accuracy_score(
    y_test,
    rf_predictions
)
```

```
xgb_accuracy = accuracy_score(
    y_test,
    xgb_predictions
)

xgb_precision = precision_score(
    y_test,
    xgb_predictions,
    average="weighted"
)

xgb_recall = recall_score(
    y_test,
    xgb_predictions,
    average="weighted"
)

xgb_f1 = f1_score(
    y_test,
    xgb_predictions,
    average="weighted"
)

xgb_confusion_matrix = confusion_matrix(
    y_test,
    xgb_predictions
)

print("Random Forest Accuracy:",
      rf_accuracy * 100)

print("XGBoost Accuracy:",
      xgb_accuracy * 100)

print("XGBoost Precision:",
      xgb_precision)

print("XGBoost Recall:",
      xgb_recall)

print("XGBoost F1 Score:",
      xgb_f1)

print("Confusion Matrix:")
print(xgb_confusion_matrix)
```

```
# --------------------------------------------------
# 8. SELECT BEST MODEL
# --------------------------------------------------

if xgb_accuracy >= rf_accuracy:
    best_model = xgb_model
    best_model_name = "XGBoost"
else:
    best_model = rf_model
    best_model_name = "Random Forest"

print("Best Model:", best_model_name)


# --------------------------------------------------
# 9. LOAD NEW STUDENT DATA
# --------------------------------------------------

new_data = pd.read_csv(
    "new_students.csv"
)

student_ids = new_data["StudentID"]
student_names = new_data["Name"]


# --------------------------------------------------
# 10. PREPARE NEW DATA FOR PREDICTION
# --------------------------------------------------

X_new = new_data.drop(
    columns=["StudentID", "Name"]
)


# --------------------------------------------------
# 11. BATCH PREDICTION
# --------------------------------------------------

predictions = best_model.predict(X_new)


# --------------------------------------------------
# 12. CREATE RESULT DATAFRAME
```

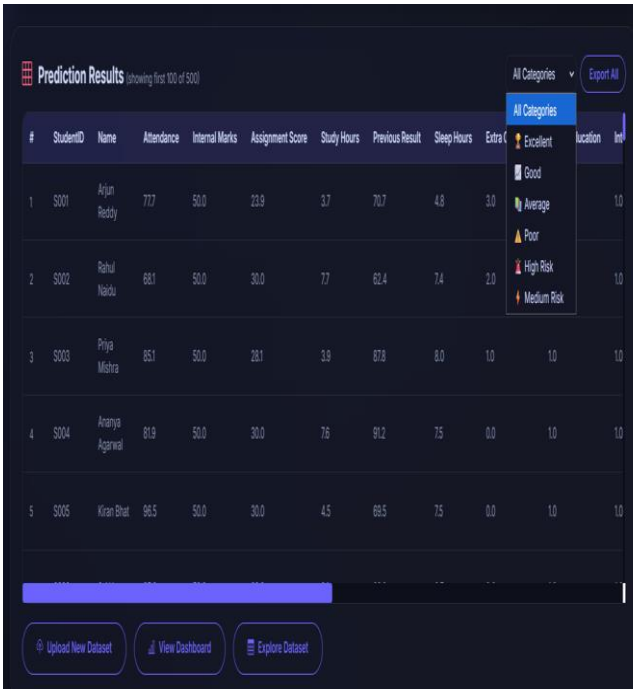
```
# --------------------------------------------------

results = pd.DataFrame({
    "StudentID": student_ids,
    "Name": student_names,
    "Predicted Performance": predictions
})


# --------------------------------------------------
# 13. RISK IDENTIFICATION
# --------------------------------------------------

def get_risk_level(performance):

    if performance == "Poor":
        return "High"

    elif performance == "Average":
        return "Medium"

    else:
        return "Low"


results["Risk Level"] = results[
    "Predicted Performance"
].apply(get_risk_level)


# --------------------------------------------------
# 14. CONTRIBUTING FACTORS
# --------------------------------------------------

def get_reasons(row):

    reasons = []

    if "Attendance" in row.index:
        if row["Attendance"] < 75:
            reasons.append("Low Attendance")

    if "Internal_Marks" in row.index:
        if row["Internal_Marks"] < 50:
            reasons.append("Low Internal Marks")
```

```
    if "Assignment_Score" in row.index:
        if row["Assignment_Score"] < 50:
            reasons.append("Low Assignment Score")

    if "Study_Hours" in row.index:
        if row["Study_Hours"] < 3:
            reasons.append("Low Study Hours")

    if len(reasons) == 0:
        return "No major contributing factor identified"

    return " + ".join(reasons)


results["Contributing Factors"] = new_data.apply(
    get_reasons,
    axis=1
)


# --------------------------------------------------
# 15. DISPLAY FINAL RESULTS
# --------------------------------------------------

print("\nFinal Prediction Results:")
print(results)


# --------------------------------------------------
# 16. SAVE RESULTS
# --------------------------------------------------

results.to_csv(
    "student_prediction_results.csv",
    index=False
)

print(
    "\nPrediction results saved successfully."
)
```

OUTPUT


StudentAI
Home
Upload
Explore
Dashboard
Compare
Predict
Results
Explore the Training Data
Browse all 500 student records, filter by performance class, and inspect feature statistics
500
TOTAL RECORDS
12
FEATURES
4
CLASSES
Average
Excellent
Good
Feature Statistics
Feature Min Max Mean Std Dev
Attendance 50.0 100.0 82.45 9.7
Internal Marks 20.0 50.0 44.45 6.31
Assignment Score 12.3 30.0 27.4 3.31
Study Hours 1.0 10.0 5.17 1.87
Previous Result 40.0 100.0 71.79 13.23
Sleep Hours 4.0 10.0 7.04 1.1
Extra Curricular 0.0 5.0 1.94 1.32
Parent Education 1.0 1.0 1.0 0.0
Internet Access 0.0 1.0 0.88 0.32
Library Usage 3.0 3.0 3.0 0.0


StudentAI
Batch Prediction Analytics
500
28
82.4%
44.5/50
Predicted Performance (Uploaded Students)
Attendance Distribution (Uploaded Students)
Internal Marks Distribution (Uploaded Students)
Study Hours Distribution (Uploaded Students)


Prediction Results
All Categories
Export All
StudentID Name Attendance Internal Marks Assignment Score Study Hours Previous Result Sleep Hours
1 S001 Arjun Reddy 77.7 50.0 23.9 3.7 70.7 4.8
2 S002 Rahul Naidu 68.1 50.0 30.0 7.7 62.4 7.4
3 S003 Priya Mishra 85.1 50.0 28.1 3.9 87.8 8.0
4 S004 Ananya Agarwal 81.9 50.0 30.0 7.6 91.2 7.5
5 S005 Kiran Bhat 96.5 50.0 30.0 4.5 69.5 7.5
All Categories
Excellent
Good
Average
Poor
High Risk
Medium Risk
Upload New Dataset
View Dashboard
Explore Dataset